

LEVERAGING MACHINE LEARNING TECHNIQUES FOR HEART DISEASE EARLY DETECTION AND DIAGNOSIS

*Report submitted to the SASTRA Deemed to be
University as the requirement for the course*

CSE300 - MINI PROJECT

Submitted by

MADHUMITHA.C

(Reg. No.: 127156090, B.Tech Computer Science and Engineering(AIDS))

SANTHOSH.LS

(Reg. No.:127156145, B. Tech Computer Science and Engineering(AIDS))

SRUTHI BAL.R

(Reg. No.:127156155, B. Tech Computer Science and Engineering(AIDS))

MAY 2026



**SCHOOL OF COMPUTING
THANJAVUR, TAMILNADU, INDIA-613401**



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

SCHOOL OF COMPUTING

THANJAVUR – 613 401

Bonafide Certificate

This is to certify that the report titled “**Leveraging Machine Learning Techniques for Heart Disease Early Detection and Diagnosis**” submitted as a requirement for the course, **CSE300 : MINI PROJECT** for B.Tech. is a bonafide record of the work done by **Ms.Madhumitha C (Reg. No.127156090, B.Tech Computer Science and Engineering(AIDS))**, **Ms.Santhoshi S (Reg.No.127156145, B.Tech Computer Science and Engineering(AIDS))**, **Sruthi Bai R (Reg.No.127156155, B.Tech Computer Science and Engineering(AIDS))** during the academic year 2025-26, in the School of Computing, under my supervision.

Signature of Project Supervisor :

Name with Affiliation : Dr. S.Vishnika Veni, Asst.Professor II,SOC

Date :

Mini Project *Viva voce* held on _____

Examiner 1

Examiner 2

Acknowledgements

We would like to thank our Honorable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honorable Vice-Chancellor **Dr. S.Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning and Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University, for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing; **Dr. R. Muthaiah**, Associate Dean, Research; **Dr. K. Ramkumar**, Associate Dean, Academics; **Dr. D. Manivannan**, Associate Dean, Infrastructure; and **Dr. R. Alageswaran**, Associate Dean, Student Welfare.

Our guide **Dr. S.Vishnika Veni** Asst.Professor II, School of Computing, was the driving force behind this project. His deep insight into the field and invaluable suggestions helped us make significant progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge the contributions and encouragement from our family and friends resulting in the successful completion of this project.

List of Figure

Figure No.	Title	Page No.
Fig 4.1	Distribution of Patients with and without Heart Disease	27
Fig 4.2	Heart Disease Percentage by Age Group	27
Fig 4.3	Age Distribution of Patients	28
Fig 4.4	Heart Disease Distribution by Exercise-Induced Angina	29
Fig 4.5	Heart Disease Distribution by Resting ECG	29
Fig 4.6	Age Distribution by Heart Disease	30
Fig 4.7	Correlation Matrix of Features	31
Fig 4.8	Feature Importance Using SHAP Values	31
Fig 4.9	Selecting Important Features and SHAP Summary Visualization	32
Fig 4.10	Confusion Matrices-Baseline Models	34
Fig 4.10.1	Support Vector Machine	34
Fig 4.10.2	ADA-Boost	35
Fig 4.10.3	XGBoost	36
Fig 4.10.4	KNN-Best K Selection	37
Fig 4.10.5	Random Forest	38
Fig 4.10.6	Extra Tree	40
Fig 4.10.7	LightGBM	41
Fig 4.10.8	Logistic Regression	42

Fig 4.10.9	Linear Discriminant Analysis	43
Fig 4.10.10	CatBoost	44

Abbreviation

ML	Machine Learning
KNN	K-Nearest Neighbors
RF	Random Forest
XGBoost	Extreme Gradient Boosting
LightGBM	Light Gradient Boosting
SHAP	Shapely Additive Explanations
CV	Cross Validation
HPT	HyperParameter Tuning
ROC-AUC	Receiver Operating Characteristic and Area Under Curve
ET or ETC	Extra Trees Classifier

ABSTRACT

Cardiovascular disease is one of the leading causes of death worldwide and remains a major public health concern. It affects millions of people each year and significantly impacts quality of life and healthcare system. The growing availability of comprehensive patient health records in hospital databases provides an opportunity to detect heart disease at an early stage, especially based on common symptoms such as chest pain, shortness of breath, fatigue, and irregular heartbeat. Machine learning techniques help reduce dependence on expensive clinical tests and laboratory investigations, thereby lowering healthcare costs for both patients and providers. The proposed CatBoost model outperforms several existing state-of-the-art methods in terms of accuracy and precision, highlighting its potential for improving early diagnosis, prevention strategies, and overall quality of life. Among multiple machine-learning models including SVM, KNN, Logistic Regression, Random Forest, Gradient Boosting, XGBoost, AdaBoost, the proposed Catboost model achieved the best performance with 90.94% accuracy and 92.31% F1-score, improving results about 2-6% compared to other classifiers, demonstrating clear enhancement in heart-disease prediction. .

KEY WORDS: Machine Learning, Heart Disease Prediction, Feature Engineering, SHAP, Classification, XGBoost, Catboost, SVM, Hypparameter Tuning, ROC-AUC.

Table of Contents

Title	Page No
Bonafide Certificate	2
Acknowledgements	4
List of Figures	5
Abbreviations	7
Abstract	8
Summary of the Base Paper	9
Merits and Demerits of the Base Paper	14
Source Code	16
Results and Discussion	27
Performance Measures	44
Conclusion and Future Plans	45
Reference	46

CHAPTER 1

SUMMARY OF THE BASE PAPER

1.1 PAPER IDENTIFICATION

Title: Advanced Machine Learning Techniques for Cardiovascular Disease Early Detection and Diagnosis

Authors: Nadiah A. Baghdadi, Sally Mohammed Farghaly Abdelallem, Amer Malki, Ibrahim Gad, Ashraf Ewis, and Elsayed Atlam

Journal: Journal of Big Data

Publisher: SpringerOpen (Springer Nature)

Year: 2023

DOI: [10.1186/s40537-023-00817-1](https://doi.org/10.1186/s40537-023-00817-1)

1.2 RESEARCH PROBLEM AND MOTIVATION

Cardiovascular disease(CVD) is one of the leading causes of death worldwide.Early diagnosis using traditional clinical methods is challenging.Large volumes of hospital data are not available. One of the major challenges in managing CVD is that patients typically do not exhibit symptoms in the early stages of the disease, making timely diagnosis extremely difficult. This creates a critical need for automated, intelligent systems capable of detecting CVD at an early stage using available patient health data. ML techniques can identify hidden patterns in medical data.ML techniques improve prediction accuracy and clinical decision support.Early detection reduces healthcare cost and improves patient outcomes.The Paper proposes machine learning techniques that hold significant potential in addressing this challenge by analyzing hospital health records to support clinical decision-making.

1.3 OBJECTIVE

The primary objectives of this study was to develop robust, efficient machine learning algorithms specifically designed for the automatic selection of key features and early detection of heart disease, to reduce the financial burden of extensive clinical investigations, and to provide reliable guidance to medical specialists for improved patient care.

1.4 DATASET DESCRIPTION

-The study utilized the Heart Failure Prediction Dataset sourced from the UCI Machine Learning Repository. This dataset was created by combining five well-known heart disease datasets — Cleveland (303 records), Hungarian (294 records), Statlog Heart (270 records), Long Beach VA (200 records), and Switzerland (123 records). After removing 272 duplicate records from the total of 1190, the final dataset contained 918 observations with 12 columns and 11 features. The features include patient age, sex, type of chest pain, resting blood pressure, cholesterol level, fasting blood sugar, resting ECG result, maximum heart rate, exercise-induced angina, ST depression (Oldpeak), and ST slope. The target variable is binary, where 1 indicates the presence of heart disease and 0 indicates absence. Among the 918 patients, 508 were diagnosed with heart disease and 410 were not.

1.5 METHODOLOGY

The proposed methodology was divided into three main phases — pre-processing, training, and classification.

In the pre-processing phase, missing cholesterol values (initially recorded as 0) were handled by grouping data based on cardiac condition and filling missing values with group means. SHAP (SHapley Additive Explanations) values, which are based on game theory, were used to identify and select the most significant features. Features with a Shapley value greater than 0.1 were selected as final predictors, resulting in 18 key features including interaction terms such as Cholesterol with ST Slope, Resting Blood Pressure with ST Slope, and Sex with ST Slope.

In the training phase, the dataset was split into 70% training and 30% testing sets. K-fold cross-validation was applied to assess model reliability. Several machine learning algorithms were trained and evaluated, including Support Vector Classifier (SVC), Naive Bayes, Bernoulli Naive Bayes, Stochastic Gradient Descent (SGD), K-Nearest Neighbor (KNN), Random Forest, Decision Tree, XGBoost, AdaBoost, Gradient Boosting, LightGBM, and CatBoost.

In the classification phase, labeled data was used as input, significant features were extracted, and the optimized model was applied to classify new patient instances as either having heart disease or not.

1.6 FEATURE IMPORTANCE

- Interaction terms are created between important clinical features such as **Cholesterol $\times$ ST_Slope**, **RestingBP $\times$ ST_Slope**, **Age $\times$ MaxHR**, and **Sex $\times$ ST_Slope** to capture the combined effect of multiple variables
- These interaction features help model complex relationships in medical data, improving prediction performance.

- Feature importance is then evaluated using **SHAP (Shapley Additive Explanations)**
- A **Gradient Boosting model** is used to compute SHAP values for all features.
- The **average SHAP value is calculated**, and features with importance ≥ 0.1 are selected
- Less important features are removed, which improves model accuracy, reduces noise, and enhances interpretability.

1.7 CLASSIFICATION ALGORITHM

1.7.1 RANDOM FOREST

Several decision trees are combined in this ensemble approach to increase accuracy. Improves generalization by using subsets of features and data that are chosen at random.

Key hyperparameters: Number of trees - More trees improve performance but increase computation time. Maximum number of decision splits per tree - Prevents overfitting by limiting tree growth. Splitting criteria - Determines how the best split is chosen. Number of predictors to sample for each split – Controls randomness and diversity among trees

1.7.2 K-NEAREST NEIGHBORS

A well-liked and user-friendly supervised learning algorithm. Classifies a test instance using the majority class of its k closest neighbors. Does well on tasks that require both classification and regression.

Key hyperparameter: Number of neighbors - Determines how many nearest points influence classification

1.7.3 SUPPORT VECTOR MACHINE

This maximizes class separation by using a hyperplane to separate data points. Strikes a balance between error minimization and margin maximization to guarantee improved accuracy.

1.7.4 ADABOOST

AdaBoost (Adaptive Boosting) is an ensemble learning technique that combines multiple weak classifiers, typically decision trees, to form a strong classifier. It works by assigning higher weights to misclassified instances and focusing more on difficult samples in subsequent iterations, thereby improving overall model accuracy.

Key hyperparameters: Number of estimators – Controls the number of weak learners used. Learning rate – Determines the contribution of each model to the final output. Base estimator depth – Controls complexity of individual trees.

1.7.5 GRADIENT BOOSTING

Gradient Boosting is an ensemble technique that builds models sequentially, where each new model corrects the errors made by previous models. It minimizes a loss function using gradient descent and is highly effective for capturing complex patterns in data.

Key hyperparameters: Number of estimators – Controls number of boosting stages. Learning rate – Determines step size in updating model. Maximum depth – Limits tree complexity to prevent overfitting. Subsample – Controls randomness and improves generalization.

1.7.6 LIGHT GRADIENT BOOSTING MACHINE (LIGHTGBM)

LightGBM is an efficient gradient boosting algorithm that builds decision trees using a leaf-wise approach. It is designed for high performance and faster training on large datasets while maintaining high accuracy. It handles large-scale data efficiently and reduces training time compared to traditional boosting methods.

Key hyperparameters: Number of estimators – Controls number of trees. Learning rate – Determines step size of learning. Maximum depth – Limits tree complexity. Number of leaves – Controls model flexibility. Subsample and colsample_bytree – Improve generalization and reduce overfitting.

1.7.7 LINEAR DISCRIMINANT ANALYSIS (LDA)

Linear Discriminant Analysis is a supervised learning algorithm used for classification by finding a linear combination of features that best separates classes. It assumes normal distribution of data and maximizes the distance between class means while minimizing variance within each class.

Key hyperparameters: Solver – Determines method for optimization. Shrinkage – Controls regularization to improve stability. Number of components – Defines number of linear discriminants used for classification.

1.7.8 LOGISTIC REGRESSION

Logistic Regression is a statistical classification model that predicts the probability of a binary outcome using a logistic (sigmoid) function. It is simple, efficient, and widely used for classification problems where the relationship between features and target is approximately linear.

Key hyperparameters: Regularization parameter (C) – Controls overfitting. Penalty – Defines type of regularization (L1 or L2). Solver – Optimization algorithm used for training. Maximum iterations – Controls convergence of the model.

1.7.9 EXTRA TREES CLASSIFIER

Extra Trees (Extremely Randomized Trees) is an ensemble learning method that builds multiple decision trees using random splits and random subsets of features. It increases model diversity and reduces variance, leading to improved generalization performance.

Key hyperparameters: Number of estimators – Controls number of trees. Maximum depth – Limits tree growth. Minimum samples split – Controls splitting of nodes. Maximum features – Determines number of features considered at each split

1.7.10 EXTREME GRADIENT BOOSTING (XGBOOST)

XGBoost is an advanced gradient boosting algorithm that improves performance using regularization, parallel computation, and efficient handling of missing values. It builds trees sequentially to minimize errors and is widely used for achieving high accuracy in classification tasks.

Key hyperparameters: Number of estimators – Number of boosting rounds. Learning rate – Controls speed of learning. Maximum depth – Controls tree complexity. Subsample and colsample_bytree – Prevent overfitting. Regularization parameters – Improve model generalization.

1.7.11. CATBOOST

CatBoost is a gradient boosting algorithm that is specifically designed to handle categorical features efficiently. It reduces the need for extensive preprocessing and minimizes overfitting through ordered boosting techniques.

Key hyperparameters: Number of iterations – Controls number of boosting rounds. Learning rate – Determines model update speed. Depth – Controls tree depth. Loss function – Defines optimization objective. L2 regularization – Prevents overfitting

1.8 RESULT

Among all the classifiers evaluated, the tuned CatBoost model delivered the best overall performance with an accuracy of 90.94%, precision of 93.17%, recall of 91.46%, and an F1-score of 92.3%. Other classifiers such as Random Forest, Logistic Regression, SVM, and KNN also performed well but with slightly lower recall and F1 scores compared to the tuned CatBoost model. Lower-performing classifiers included XGBoost and AdaBoost, which showed moderate accuracy but relatively lower recall values.

CHAPTER 2

MERITS AND DEMERITS OF THE BASE PAPER

LITERATURE SURVEY:

1) Ensemble learning with explainable AI for improved heart disease prediction based on multiple datasets Shahid Mohammad Ganie, Pijush Kanti Dutta Pramanik & Zhongming Zhao (2025)

Stacking + voting Ensemble on combined datasets with SHAP explainability

2) Heart Risk Failure Prediction Using a Novel Feature Selection Method for Feature Refinement and Neural Network for Classification Ashir Javeed, Sanam Shahla Rizvi, Shijie Zhou, Rabia Riaz, Shafqat Ullah Khan, Se Jin Kwon (2020)

Novel feature selection + Artificial Neural Network (ANN) .

3) A Heart Disease Prediction Model using SVM-Decision Trees-Logistic Regression (SDL) by Mythili T., Dev Mukherji, Nikita Padalia, Abhiram Naidu (2020) [3]

Compared SVM, Decision Tree, Logistic Regression .

4) Medical decision making diagnosis system integrating k-means and Naïve Bayes algorithms Aigerim Altayeva; Suleimenov Zharas; Young Im Cho (2020) [4]

K-means clustering + Naive Bayes classification .

5) An Explainable and Accurate Machine Learning Approach for Early Heart Disease Prediction Using Feature Selection and Ensemble Techniques Authors Khaliq Ahmed (2025) [5]

Feature selection + Ensemble ML (RF,GB,XGBoost) SMOTE with XAI .

6) Explainable AI assisted heart disease diagnosis through effective feature engineering and stacked ensemble learning panel Partho Ghose d, Khondokar Oliullah a, Md Kawsher Mahbub b, Milon Biswas c, Kazi Nur Uddin b, Hasan M. Jamil (2025) [6]

Decision Tree, Random Forest, Logistic Regression .

MERITS:

- **Effective Feature Engineering:** Introduced interaction terms between clinical features.Helps capture combined effects (very important in medical data).
- **Use of SHAP for Feature Selection:** Used SHAP values for feature importance.Improves interpretability of the model.Helps select only relevant features(>0.1 threshold).
- **Comparison of Multiple Models:** Evaluated models like Random Forest,AdaBoost,GradientBoosting,Extra Trees,SVN,KNN,CatBoost etc.
- **Good Performance:**Achieved high accuracy and AUC(~0.92+).Shows strong predictive capability

DEMERITS:

- **Excessive Computational Cost for Certain Models:**SVMs are highly computationally expensive in terms of training time and hence are not ideal for real-time or large-scale applications.
- **Poor Performance with Some Instance Selection Techniques:**KNN algorithms, for example, gave poor results when trained on datasets selected by BPLSH, and there were sharp declines in accuracy and F1 score.
- **Difficulty in Phishing URL Detection:**Phishing URLs always yielded lower true positive rates, and this category was still difficult to classify correctly.
- **Class Imbalance Problems:**Most datasets have significantly more benign URLs than malicious ones. This imbalance biases learning, necessitating extra preprocessing such as undersampling, which can decrease overall data diversity.
- **Limited Contextual and Content-Based Analysis:**Most models use lexical and structural properties of URLs, which may overlook threats only identifiable through content or behavioral analysis.
- **Less Generalization to Emerging Threats:**While machine learning learns from previous patterns, zero-day or new attacks might not be identified if the model is not updated on a regular basis.

CHAPTER 3

SOURCE CODE

3.1 Importing Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, confusion_matrix, ConfusionMatrixDisplay, classification_report

import shap

from sklearn.svm import SVC

from sklearn.ensemble import AdaBoostClassifier

from sklearn.ensemble import GradientBoostingClassifier

from xgboost import XGBClassifier

from sklearn.neighbors import KNeighborsClassifier

from catboost import CatBoostClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.linear_model import SGDClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier
```



```

from sklearn.ensemble import ExtraTreesClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

import lightgbm as lgb

```

3.2 Loading The Dataset

```

df=pd.read_csv('heart.csv')
target='HeartDisease'
print('Dataset Shape:', df.shape)
print('All Columns:', list(df.columns))

```

3.3 Handling Missing Values Using Group-wise Mean Imputation

```

df['Cholesterol']=df.groupby(target)['Cholesterol'].transform(lambda
x:x.replace(0,x.mean()))

df['RestingBP']=df.groupby(target)['RestingBP'].transform(lambda x:x.replace(0,x.mean()))

```

3.4 Converting Categorical Variables into Dummy Variables (One-Hot Encoding)

```

df=pd.get_dummies(df,drop_first=True)

```

3.5 Creating Interaction Features Between Variables for Better Model Performance

```

df["Cholesterol_ST_Slope_Flat"] = df["Cholesterol"] * df["ST_Slope_Flat"]

df["RestingBP_ST_Slope_Up"] = df["RestingBP"] * df["ST_Slope_Up"]

df["Sex_M_ST_Slope_Flat"] = df["Sex_M"] * df["ST_Slope_Flat"]

df["ChestPainType_ATA_RestingECG_Normal"] = (df["ChestPainType_ATA"] *
df["RestingECG_Normal"])

df["Cholesterol_Sex_M"] = df["Cholesterol"] * df["Sex_M"]

df["Oldpeak_Sex_M"] = df["Oldpeak"] * df["Sex_M"]

df["RestingBP_ST_Slope_Flat"] = df["RestingBP"] * df["ST_Slope_Flat"]

```

```

df["Cholesterol_RestingECG_Normal"] = (df["Cholesterol"] * df["RestingECG_Normal"])

df["Age_Sex_M"] = df["Age"] * df["Sex_M"]

df["RestingBP_MaxHR"] = df["RestingBP"] * df["MaxHR"]

df["Age_MaxHR"] = df["Age"] * df["MaxHR"]

df["ChestPainType_ATA_ST_Slope_Up"] = ( df["ChestPainType_ATA"] * df["ST_Slope_Up"])

df["ChestPainType_NAP_ST_Slope_Up"] = (df["ChestPainType_NAP"] * df["ST_Slope_Up"])

df["RestingBP_Oldpeak"] = df["RestingBP"] * df["Oldpeak"]

df["RestingBP_Cholesterol"] = df["RestingBP"] * df["Cholesterol"]

df["Oldpeak_ExerciseAngina_Y"] = ( df["Oldpeak"] * df["ExerciseAngina_Y"])

df["Sex_M_ST_Slope_Up"] = df["Sex_M"] * df["ST_Slope_Up"]

df["Sex_M_ST_Slope_Flat"] = df["Sex_M"] * df["ST_Slope_Flat"]

df["Oldpeak_Sex_M"] = df["Oldpeak"] * df["Sex_M"]

df["FastingBS_ST_Slope_Up"] = df["FastingBS"] * df["ST_Slope_Up"]

df["FastingBS_ST_Slope_Flat"] = df["FastingBS"] * df["ST_Slope_Flat"]

```

3.7 Separating Features and Target Variable for Model Training

```

X = df.drop("HeartDisease", axis=1)
y = df["HeartDisease"]
x=df.drop(target,axis=1)
y=df[target]
feature_names=x.columns

```

3.8 Splitting Dataset and Applying Feature Scaling

```

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=42,stratify=y)
scaler=StandardScaler()
x_train_scaled=scaler.fit_transform(x_train)
x_test_scaled=scaler.transform(x_test)

```

3.9 Feature Selection Using SHAP Values with Gradient Boosting

```

gb=GradientBoostingClassifier(random_state=42)
gb.fit(x_train_scaled,y_train)

explainer=shap.TreeExplainer(gb)
shap_values=explainer.shap_values(x_train_scaled)
importance=np.abs(shap_values).mean(axis=0)

selected_mask=importance >= 0.1
selected_features=feature_names[selected_mask]
print("Selected features")
for f in selected_features:
    print(f)

```

3.10 Selecting Important Features and Visualizing with SHAP Summary Plot

```

x_train_sel=x_train_scaled[:,selected_mask]
x_test_sel=x_test_scaled[:,selected_mask]
shap.summary_plot(shap_values,x_train_scaled,feature_names=feature_names)

```

3.11 BASELINE MODELS

3.11.1 Support Vector Machine

```

from sklearn.svm import SVC

svm = SVC(
    C=300,
    kernel='rbf',
    gamma=0.005,
    probability=True,
    class_weight='balanced',
    random_state=42)

y_pred = svm.predict(x_test_sel)
y_prob = svm.predict_proba(x_test_sel)[:,:1]
svm.fit(x_train_sel, y_train)

print("Accuracy:", accuracy_score(y_test, y_pred))

```

```

print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, y_prob))
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot()

```

3.11.2 ADABOOST

```

ada = AdaBoostClassifier(random_state=42)
param_grid = {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.1, 1]
}
grid_ada = GridSearchCV(ada, param_grid, scoring=scoring, cv=cv, n_jobs=-1)
grid_ada.fit(x_train_sel, y_train)

print("Best Params:", grid_ada.best_params_)

```

```

ada_final=AdaBoostClassifier(
    learning_rate=1,
    n_estimators=100,
    random_state=42)
ada_final.fit(x_train_sel,y_train)
ada_pred=ada_final.predict(x_test_sel)

print("Accuracy:", accuracy_score(y_test, ada_pred))
print("Precision:", precision_score(y_test, ada_pred))
print("Recall:", recall_score(y_test, ada_pred))
print("F1 Score:", f1_score(y_test, ada_pred))
print("ROC AUC:", roc_auc_score(y_test, ada_pred))
cm=confusion_matrix(y_test,ada_pred)
print("Confusion_matrix")
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Disease','No
Disease'])
disp.plot()

```

3.11.3 XGBoost

```

xgb=XGBClassifier(random_state=42,use_label_encoder=False,eval_metric='logloss')

```

```

param_grid={
    'n_estimators':[100,200],
    'max_depth':[3,5],
    'learning_rate':[0.01,0.1],
    'subsample':[0.8,1],
    'colsample_bytree':[0.8,1]
}
grid_xgb=GridSearchCV(xgb,param_grid,scoring=scoring,cv=cv,n_jobs=-1)
grid_xgb.fit(x_train_sel,y_train)
print("Best Params:",
      grid_xgb.best_params_)

xgb_final = XGBClassifier(
    colsample_bytree=0.8,
    learning_rate=0.1,
    max_depth=3,
    n_estimators=100,
    subsample=0.8,
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss')
xgb_final.fit(x_train_sel, y_train)
y_pred = xgb_final.predict(x_test_sel)
y_prob = xgb_final.predict_proba(x_test_sel)[:,:1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, y_prob))
cm=confusion_matrix(y_test,y_pred)
print("Confusion_matrix")
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Disease','No
Disease'])
disp.plot()

```

3.11.4 KNN with Best-K value

```

k_values = range(1, 21)
accuracy_scores = []
for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(x_train_sel, y_train)
    y_pred = model.predict(x_test_sel)
    acc = accuracy_score(y_test, y_pred)

```

```

    accuracy_scores.append(acc)
best_k=accuracy_scores.index(max(accuracy_scores))
print(best_k)
knn = KNeighborsClassifier(
    n_neighbors=10,
    weights='distance',
    metric='euclidean')

knn.fit(x_train_sel, y_train)
y_pred = knn.predict(x_test_sel)
y_prob = knn.predict_proba(x_test_sel)[:,1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, y_prob))

print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot()

```

3.11.5 RandomForest

```

rf = RandomForestClassifier(random_state=42)
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10],
    'min_samples_split': [2, 5],
    'max_features': ['sqrt']}
grid_rf = GridSearchCV(rf, param_grid, scoring=scoring, cv=cv, n_jobs=-1)
grid_rf.fit(x_train_sel, y_train)

print("Best Params:", grid_rf.best_params_)

rf_final=RandomForestClassifier(
    random_state=42,
    max_depth=None,
    max_features='sqrt',
    min_samples_split=2,
    n_estimators=100)
rf_final.fit(x_train_sel,y_train)
rf_pred=rf_final.predict(x_test_sel)

print("Accuracy:", accuracy_score(y_test, rf_pred))
print("Precision:", precision_score(y_test, rf_pred))

```

```

print("Recall:", recall_score(y_test, rf_pred))
print("F1 Score:", f1_score(y_test, rf_pred))
print("ROC AUC:", roc_auc_score(y_test, rf_pred))
cm=confusion_matrix(y_test,rf_pred)
print("Confusion_matrix")
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Disease','No
Disease'])
disp.plot()

```

3.11.6 Extra Tree

```

extra = ExtraTreesClassifier(random_state=42)
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10],
    'min_samples_split': [2, 5]}

grid_extra = GridSearchCV(extra, param_grid, scoring=scoring, cv=cv, n_jobs=-1)
grid_extra.fit(x_train_sel, y_train)

print("Best Params:", grid_extra.best_params_)

extra_final=ExtraTreesClassifier(
    max_depth=10,
    min_samples_split=2,
    n_estimators=200,
    random_state=42)
extra_final.fit(x_train_sel,y_train)
e_pred=extra_final.predict(x_test_sel)

print("Accuracy:", accuracy_score(y_test, e_pred))
print("Precision:", precision_score(y_test, e_pred))
print("Recall:", recall_score(y_test, e_pred))
print("F1 Score:", f1_score(y_test, e_pred))
print("ROC AUC:", roc_auc_score(y_test, e_pred))
cm=confusion_matrix(y_test,e_pred)
print("Confusion_matrix")
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Disease','No
Disease'])
disp.plot()

```

3.11.7 Light GBM

```

lgb = LGBMClassifier(random_state=42)

```

```

param_grid = {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.1],
    'max_depth': [-1, 5, 10],
    'num_leaves': [31, 50]}
grid_lgb = GridSearchCV(lgb, param_grid, scoring=scoring, cv=cv, n_jobs=-1)
grid_lgb.fit(x_train_sel, y_train)

print("Best Params:", grid_lgb.best_params_)

light_final=LGBMClassifier(
    random_state=42,
    n_estimators=200,
    learning_rate=0.1,
    max_depth=-1,
    num_leaves=31)
light_final.fit(x_train_sel,y_train)
lgbm_pred=light_final.predict(x_test_sel)

print("Accuracy:", accuracy_score(y_test, lgbm_pred))
print("Precision:", precision_score(y_test, lgbm_pred))
print("Recall:", recall_score(y_test, lgbm_pred))
print("F1 Score:", f1_score(y_test, lgbm_pred))
print("ROC AUC:", roc_auc_score(y_test, lgbm_pred))
cm=confusion_matrix(y_test,lgbm_pred)
print("Confusion_matrix")
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Disease','No
Disease'])
disp.plot()

```

3.11.8 Logistic Regression

```

log_reg = LogisticRegression(
    max_iter=1000,
    solver='lbfgs')

```



```

log_reg.fit(x_train_scaled, y_train)

y_pred_lr = log_reg.predict(x_test_scaled)

acc = accuracy_score(y_test, y_pred_lr)
prec = precision_score(y_test, y_pred_lr)
rec = recall_score(y_test, y_pred_lr)
f1 = f1_score(y_test, y_pred_lr)
print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)
cm=confusion_matrix(y_test,y_pred_lr)
disp=ConfusionMatrixDisplay(cm)
print("Confusion Matrix")
disp.plot()
print("Classification Report")
print(classification_report(y_test,y_pred_lr))

```

3.11.9 Linear Discriminant Analysis

```

lda = LinearDiscriminantAnalysis()
lda.fit(x_train_scaled, y_train)
y_pred_ld = extra_trees.predict(x_test_scaled)

acc = accuracy_score(y_test, y_pred_ld)
prec = precision_score(y_test, y_pred_ld)
rec = recall_score(y_test, y_pred_ld)
f1 = f1_score(y_test, y_pred_ld)
print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)

cm=confusion_matrix(y_test,y_pred_ld)
disp=ConfusionMatrixDisplay(cm)
print("Confusion Matrix")
disp.plot()

```

```
print("Classification Report")
print(classification_report(y_test,y_pred_ld))
```

3.11.10 CATBOOST

```
cat = CatBoostClassifier(
    verbose=0,
    random_state=42)
param_grid = {
    'iterations': [300, 500],
    'depth': [5, 6],
    'learning_rate': [0.03, 0.05],
    'l2_leaf_reg': [3, 5, 7],
    'bagging_temperature': [0, 1],
    'border_count': [64, 128]}

grid_cat = GridSearchCV(
    estimator=cat,
    param_grid=param_grid,
    scoring='f1',
    cv=5,
    n_jobs=-1)
grid_cat.fit(x_train_sel, y_train)

print("Best Params:", grid_cat.best_params_)

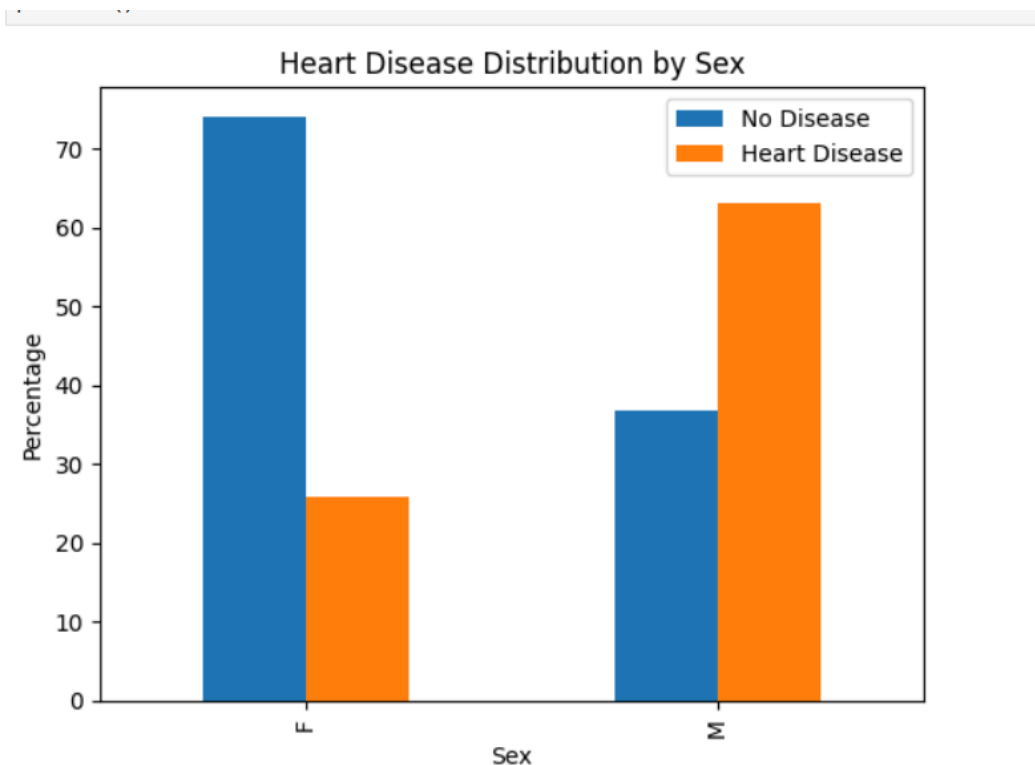
cat_final=CatBoostClassifier(
    random_state=42,
    iterations=300,
    l2_leaf_reg=7,
    bagging_temperature=0,
    border_count=128,
    depth=5,
    learning_rate=0.03)
cat_final.fit(x_train_sel,y_train)
cat_pred=cat_final.predict(x_test_sel)
print("Accuracy:", accuracy_score(y_test, cat_pred))
print("Precision:", precision_score(y_test, cat_pred))
print("Recall:", recall_score(y_test, cat_pred))
print("F1 Score:", f1_score(y_test, cat_pred))
print("ROC AUC:", roc_auc_score(y_test, cat_pred))
cm=confusion_matrix(y_test,cat_pred)
print("Confusion_matrix")
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=['Disease','No
Disease'])
disp.plot()
```

CHAPTER 4

RESULTS AND DISCUSSION

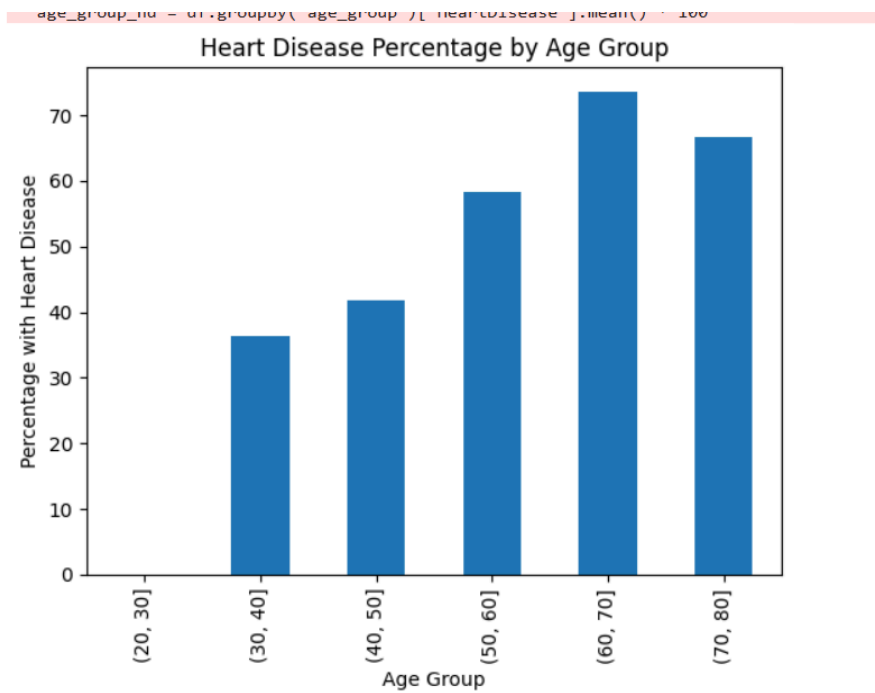
4.1 Distribution of Patients with and without Heart Disease

The bar chart illustrates the distribution of patients with and without heart disease in the dataset. It provides a clear comparison of the two classes, helping to identify whether the data is balanced or imbalanced. This visualization is useful for understanding the overall dataset composition before building machine learning models.



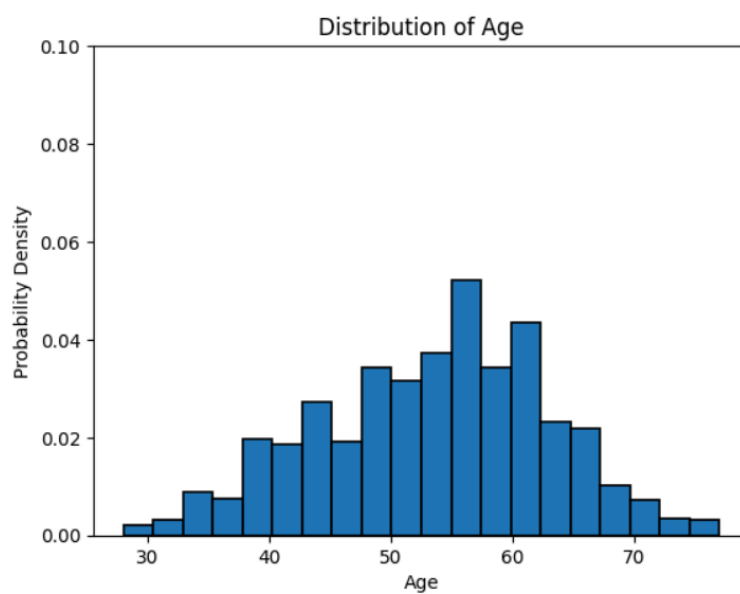
4.2 Heart Disease Percentage by Age Group

This bar chart shows the percentage of people with heart disease in different age groups. It helps to understand how heart disease risk changes with age.



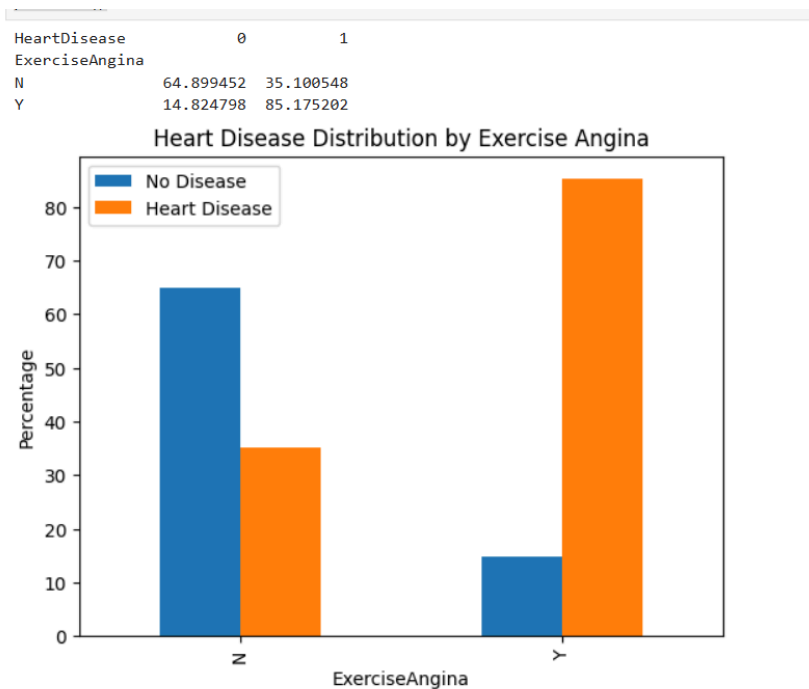
4.3 Age Distribution of Patients

This histogram shows how the ages of patients are distributed in the dataset. It helps to understand which age ranges have more patients. The plot uses probability density, so it shows the relative frequency of ages rather than exact counts.



4.4 Heart Disease Distribution by Exercise-Induced Angina

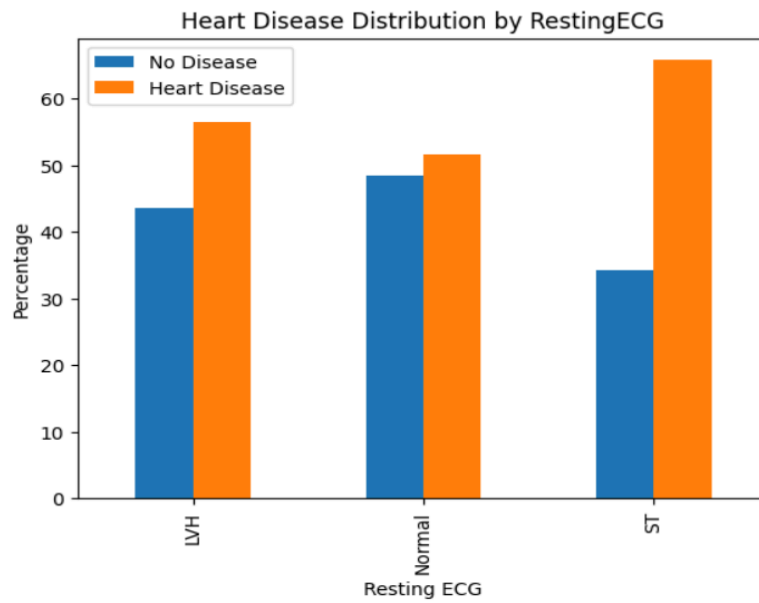
This bar chart shows the percentage of patients with and without heart disease based on whether they experience exercise-induced angina. The values are calculated as percentages within each group, making it easy to compare the distribution. This visualization helps identify the relationship between exercise angina and the likelihood of heart disease.



4.5 Heart Disease Distribution by Resting ECG

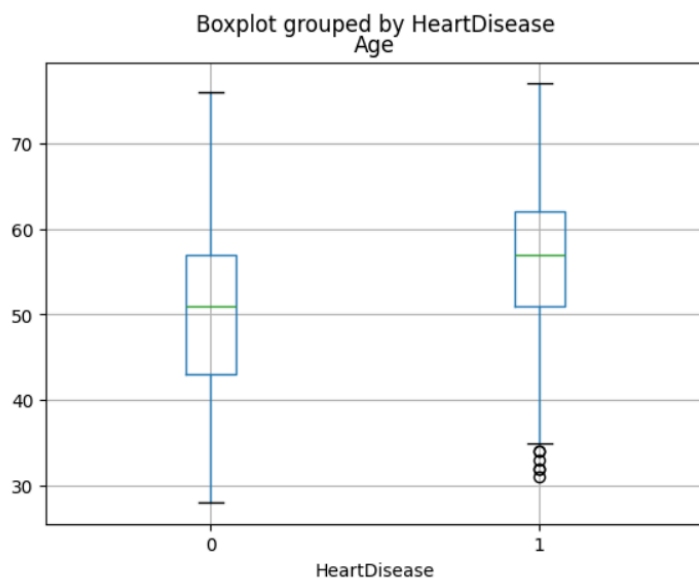
This bar chart shows the percentage of patients with and without heart disease based on their resting ECG results. The values are displayed as percentages within each ECG category, making comparison easier. This visualization helps understand the relationship between ECG results and the presence of heart disease.

HeartDisease	0	1
RestingECG		
LVH	43.617021	56.382979
Normal	48.369565	51.630435
ST	34.269663	65.730337



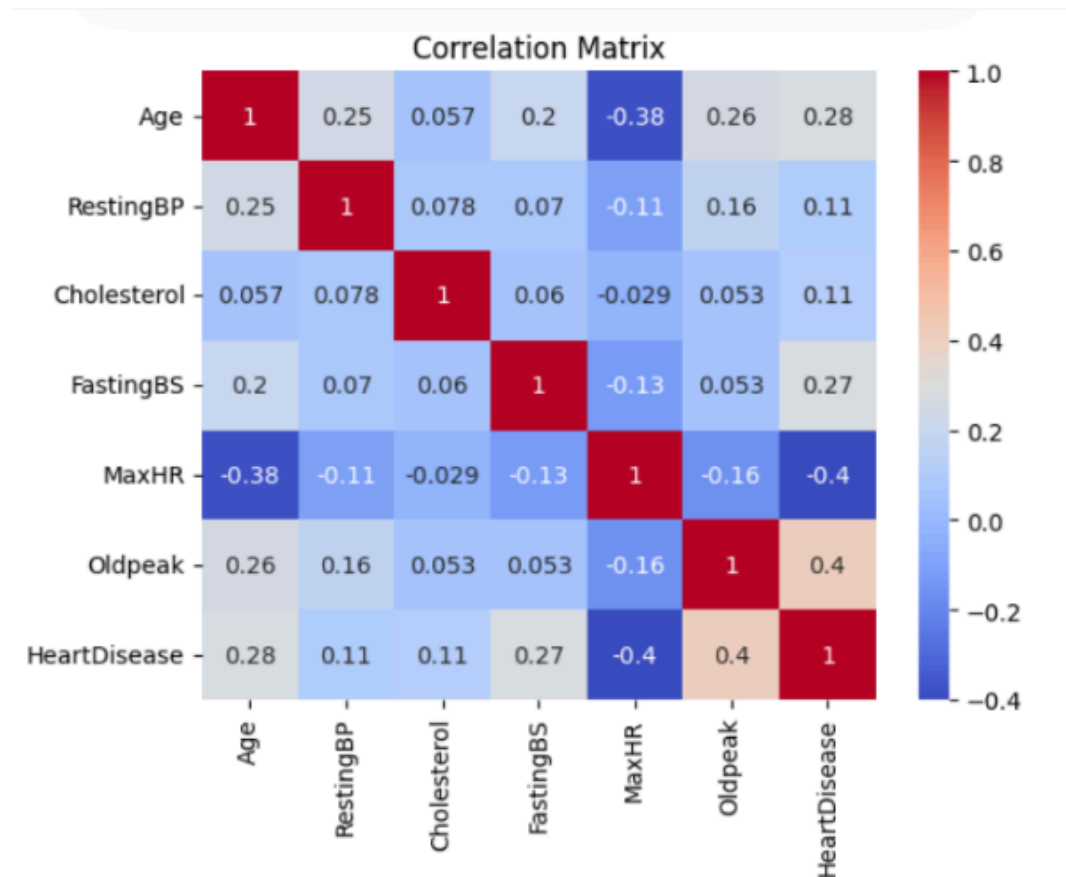
4.6 Age Distribution by Heart Disease

This box plot shows the distribution of age for patients with and without heart disease. It helps compare the median age, spread, and presence of outliers between the two groups, giving insight into how age relates to heart disease.



4.7 Correlation Matrix of Features

This heatmap shows the correlation between different variables in the dataset. It helps identify how strongly features are related to each other, with positive and negative relationships indicated by different colors. This visualization is useful for understanding feature relationships and detecting multicollinearity.



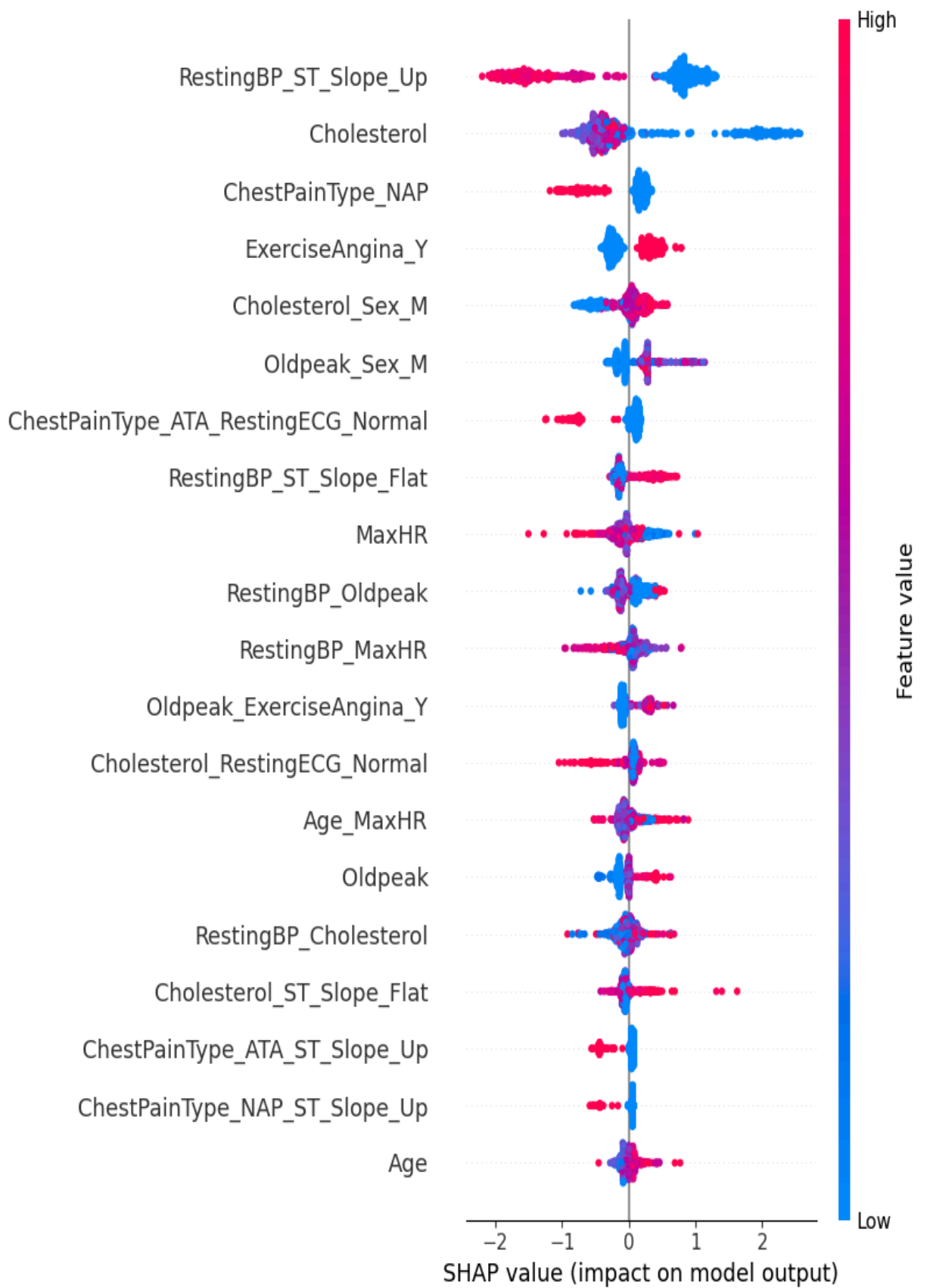
4.8 Feature Importance Using SHAP Values

This output shows the importance of each feature based on SHAP values. It calculates the average impact of each feature on the model's predictions and ranks them accordingly. This helps identify which features contribute the most to the model.

	Feature	Importance
16	RestingBP_ST_Slope_Up	0.820259
19	Cholesterol_Sex_M	0.438349
2	Cholesterol	0.392089
7	ChestPainType_NAP	0.298306
14	ExerciseAngina_Y	0.242024
20	Oldpeak_Sex_M	0.220648
17	Sex_M_ST_Slope_Flat	0.211710
28	RestingBP_Oldpeak	0.210823
21	RestingBP_ST_Slope_Flat	0.198686
6	ChestPainType_ATA	0.171923
18	ChestPainType_ATA_RestingECG_Normal	0.154983
4	MaxHR	0.124786
22	Cholesterol_RestingECG_Normal	0.121903
23	Age_Sex_M	0.119268
30	Oldpeak_ExerciseAngina_Y	0.110344
5	Oldpeak	0.106508
15	Cholesterol_ST_Slope_Flat	0.100623
33	FastingBS_ST_Slope_Flat	0.095964
24	RestingBP_MaxHR	0.085836
3	FastingBS	0.080298
0	Age	0.071307
25	Age_MaxHR	0.070471
12	ST_Slope_Up	0.068612
29	RestingBP_Cholesterol	0.064361
26	ChestPainType_ATA_ST_Slope_Up	0.057643
8	ChestPainType_TA	0.055215
1	RestingBP	0.032688
13	Sex_M	0.027330
27	ChestPainType_NAP_ST_Slope_Up	0.027110
11	ST_Slope_Flat	0.016549
9	RestingECG_Normal	0.002139
10	RestingECG_ST	0.000000
31	Sex_M_ST_Slope_Up	0.000000
32	FastingBS_ST_Slope_Up	0.000000

4.9 Selecting Important Features and SHAP Summary Visualization

This step selects the most important features based on a threshold of SHAP importance values. Only features with higher contribution are retained for further analysis. The SHAP summary plot visualizes the impact of all features on the model's predictions, helping to understand feature influence.



4.10 Confusion Matrices – Baseline Models

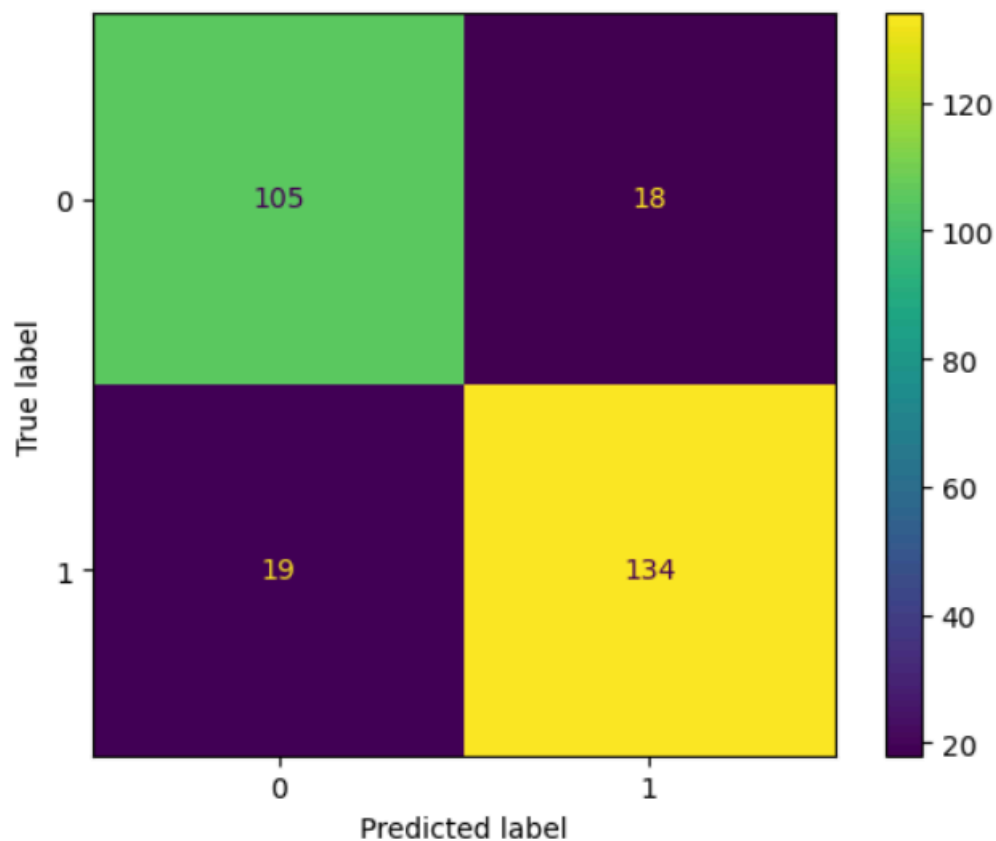
4.10.1 Support Vector Machine

SVM (Support Vector Machine) is a machine learning algorithm used for classification. It separates data into different classes by drawing a boundary (line or hyperplane). It works well for complex and high-dimensional data.

```
Accuracy: 0.8623188405797102
Precision: 0.891156462585034
Recall: 0.8562091503267973
F1 Score: 0.8733333333333333
ROC AUC: 0.9221531431000585
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.87	0.85	123
1	0.89	0.86	0.87	153
accuracy			0.86	276
macro avg	0.86	0.86	0.86	276
weighted avg	0.86	0.86	0.86	276

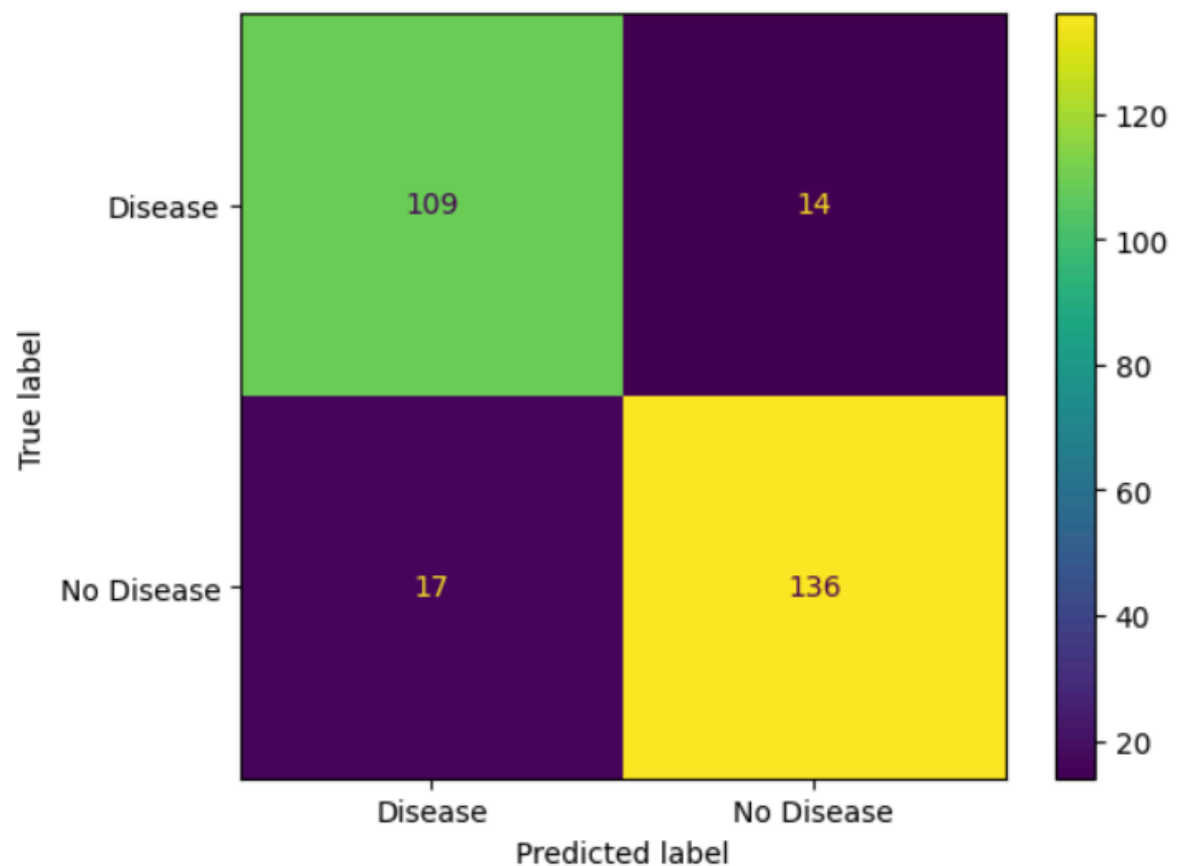


4.10.2 ADABOOST

AdaBoost (Adaptive Boosting) is an ensemble machine learning algorithm used for classification tasks. It works by combining multiple weak learners, typically decision trees, to create a strong predictive model. AdaBoost improves accuracy by focusing more on incorrectly classified instances during each iteration.

```
Accuracy: 0.8876811594202898  
Precision: 0.9066666666666666  
Recall: 0.8888888888888888  
F1 Score: 0.8976897689768977  
ROC AUC: 0.8875338753387534  
Confusion_matrix
```

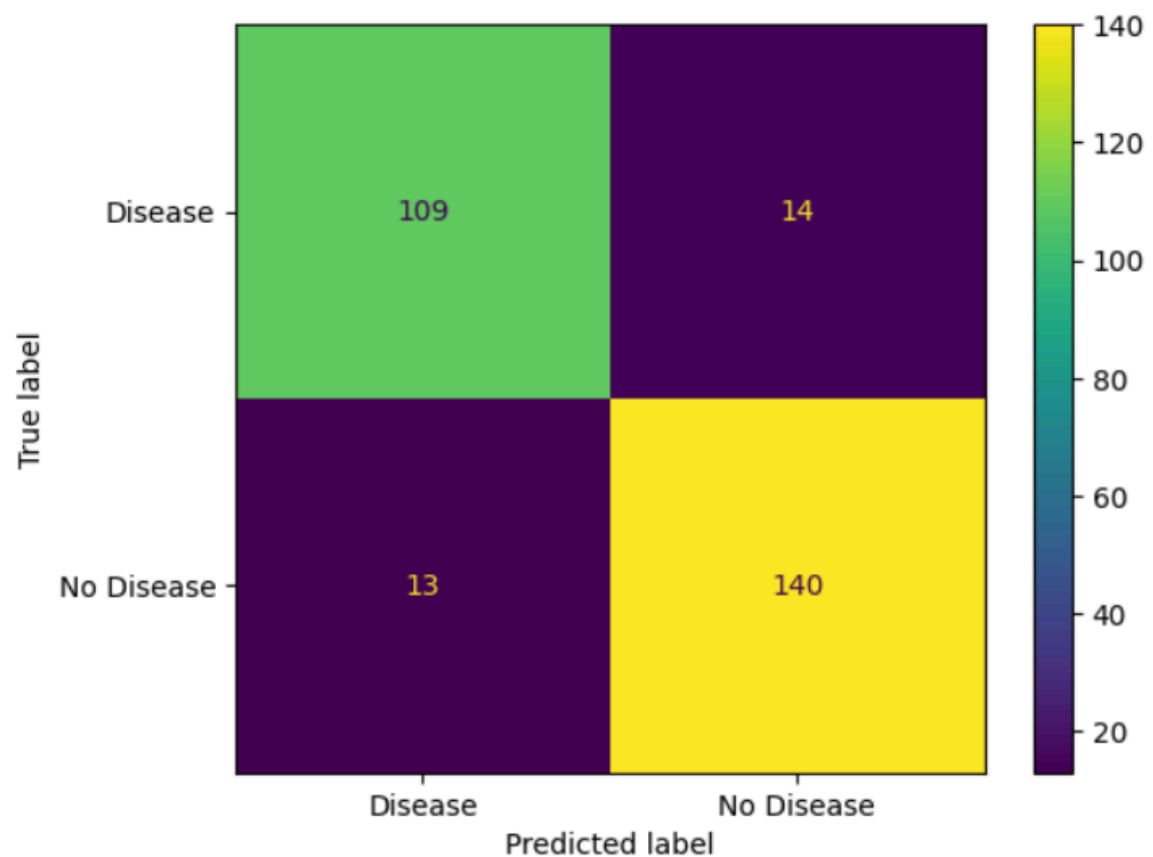
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1b1a4f1ee00>
```



4.10.3 XGBoost

XGBoost is a powerful machine learning algorithm used for classification and regression. It builds multiple decision trees and combines them to improve accuracy. It is known for its speed and high performance on large datasets.

```
Accuracy: 0.9021739130434783  
Precision: 0.9090909090909091  
Recall: 0.9150326797385621  
F1 Score: 0.9120521172638436  
ROC AUC: 0.9525479568521176  
Confusion_matrix  
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1b1a50bb190>
```



4.10.4 KNN- Best K Selection

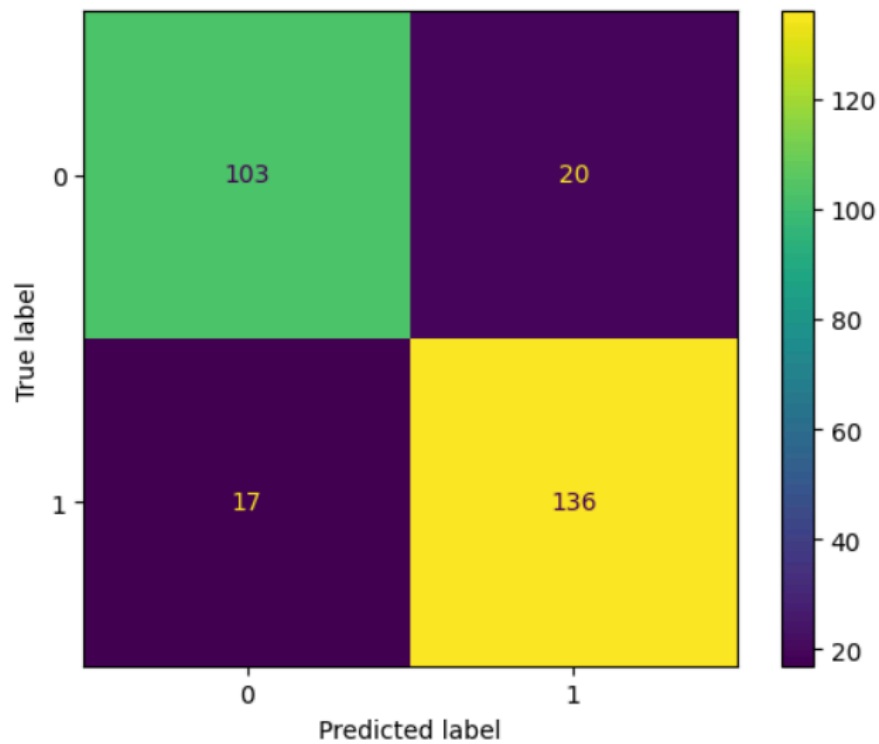
KNN (K-Nearest Neighbors) is a simple machine learning algorithm used for classification and regression. It classifies data based on the nearest neighboring data points. The output depends on the majority class of its nearest neighbors.

```
Accuracy: 0.8659420289855072  
Precision: 0.8717948717948718  
Recall: 0.8888888888888888  
F1 Score: 0.8802588996763754  
ROC AUC: 0.9324087358520645
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.86	0.84	0.85	123
1	0.87	0.89	0.88	153
accuracy			0.87	276
macro avg	0.87	0.86	0.86	276
weighted avg	0.87	0.87	0.87	276

```
: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7ac56f1e4500>
```

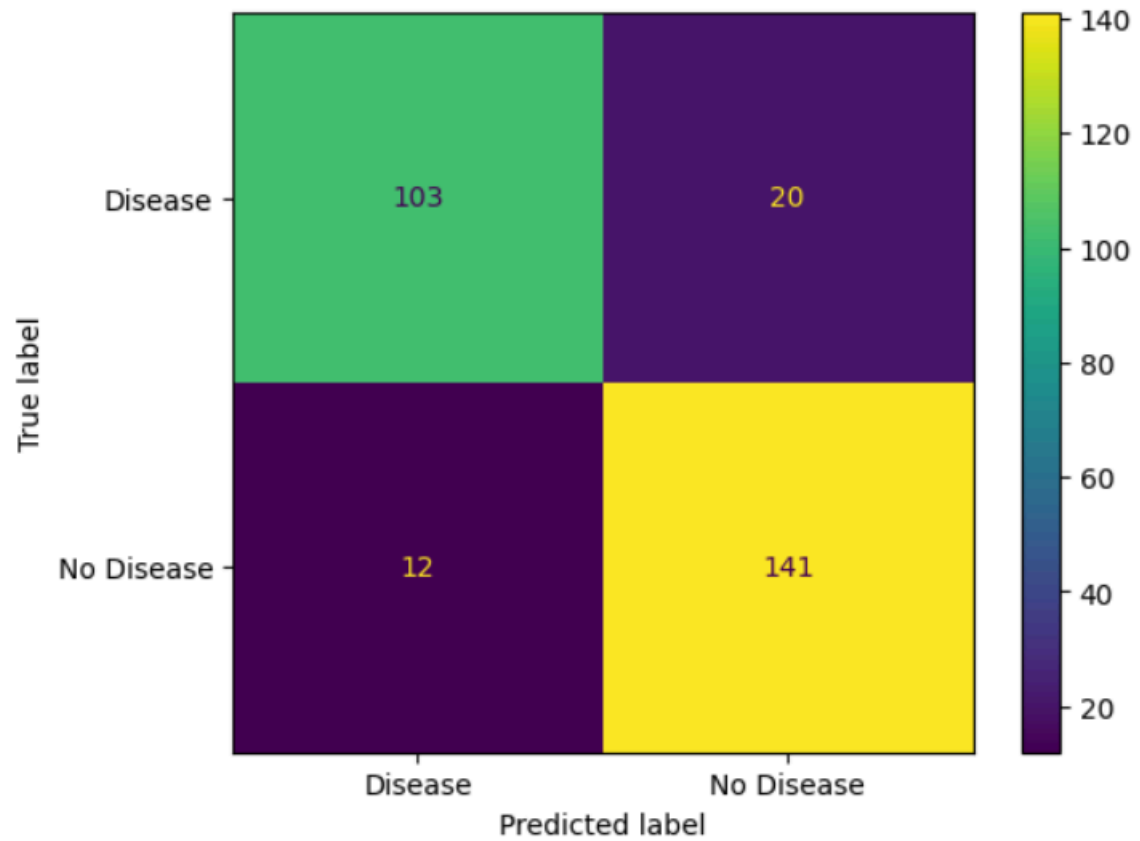


4.10.5 Random Forest

Random Forest is a machine learning algorithm used for classification and regression. It builds multiple decision trees and combines their results for better accuracy. It helps reduce overfitting and improves model performance.

Accuracy: 0.8840579710144928
Precision: 0.8757763975155279
Recall: 0.9215686274509803
F1 Score: 0.8980891719745223
ROC AUC: 0.87948350071736
Confusion_matrix

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1b1a1396c80>

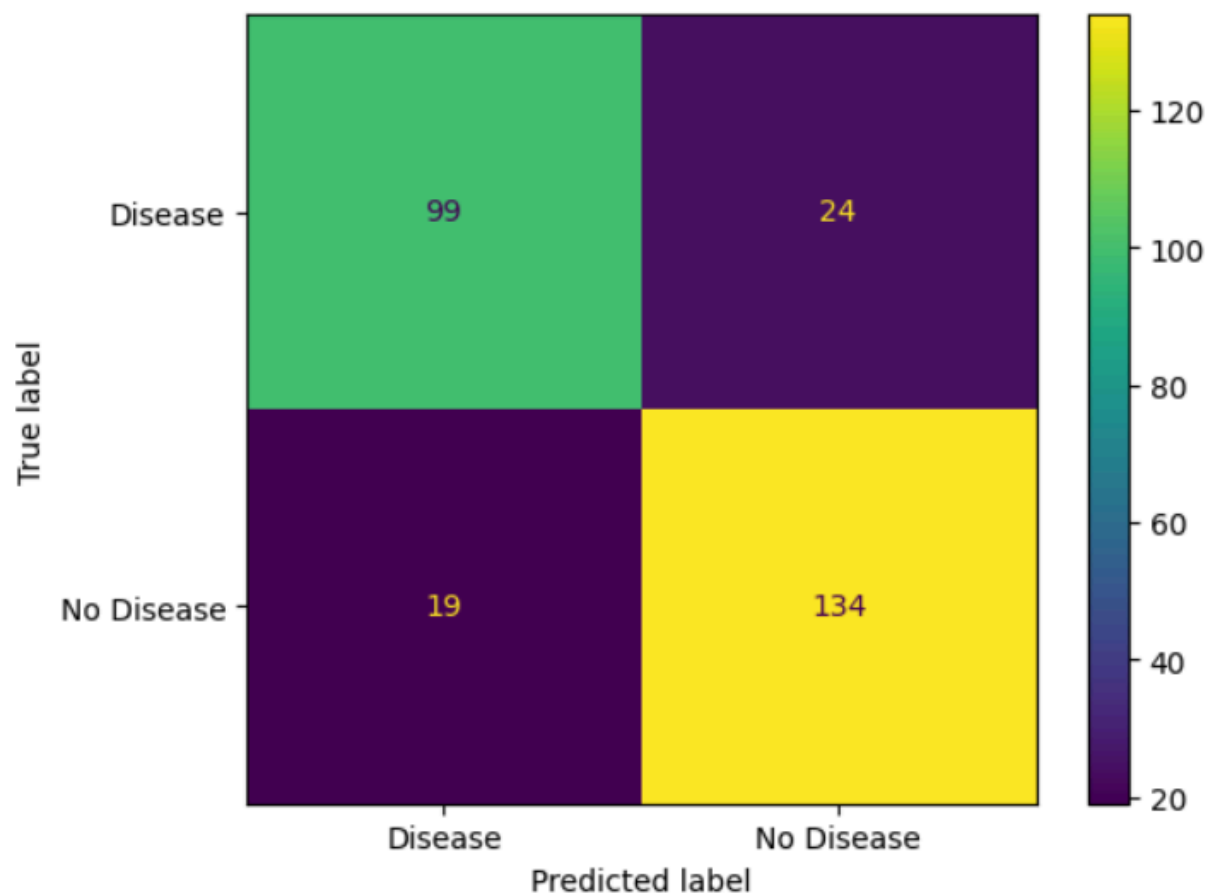


4.10.6 Extra Tree

Extra Trees (Extremely Randomized Trees) is a machine learning algorithm used for classification and regression. It builds multiple decision trees using random splits of data. This randomness helps improve accuracy and reduces overfitting.

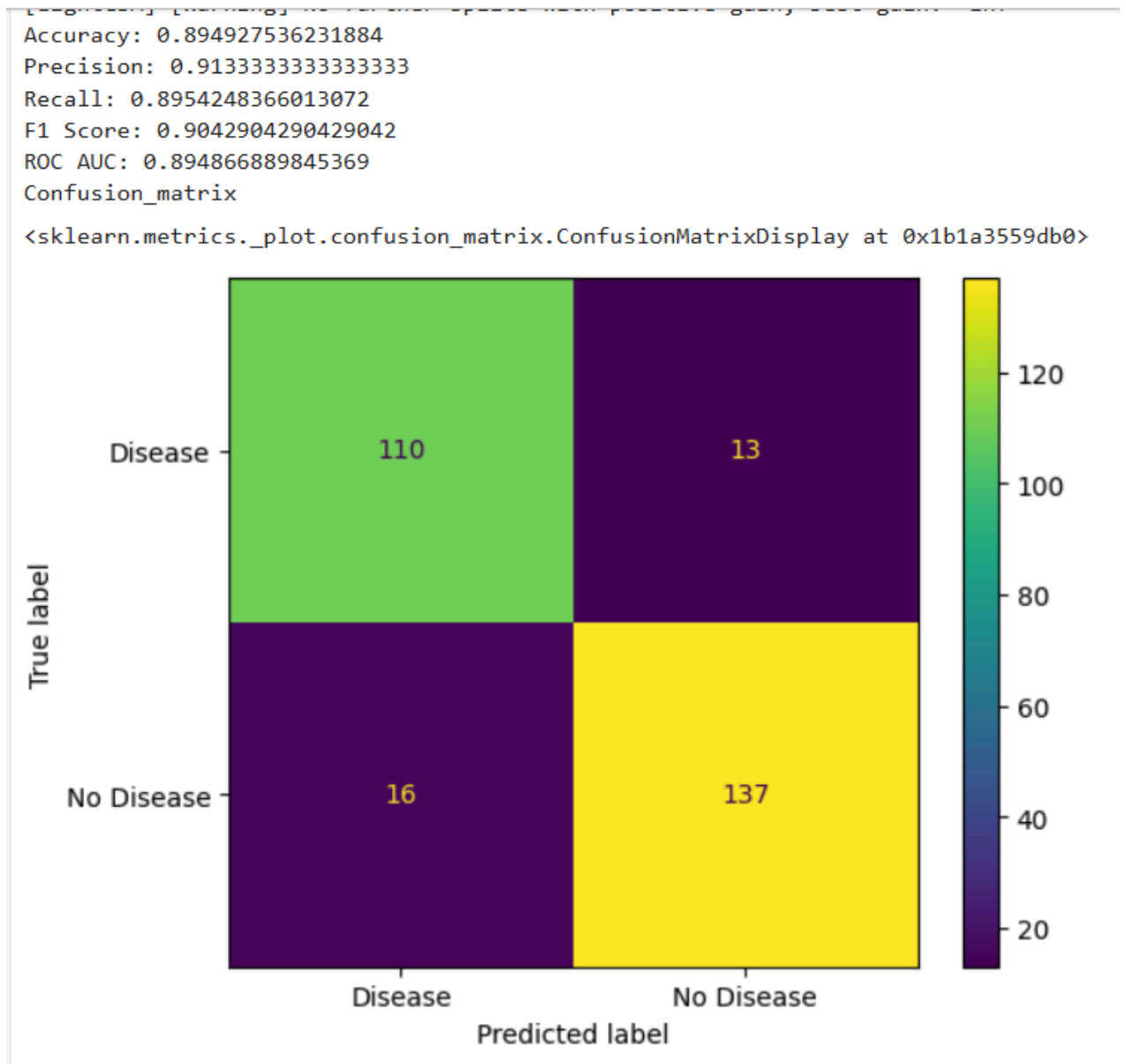
```
Accuracy: 0.8442028985507246
Precision: 0.8481012658227848
Recall: 0.8758169934640523
F1 Score: 0.8617363344051447
ROC AUC: 0.8403475211222702
Confusion_matrix
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1b1a13bdc30>
```



4.10.7 LightGBM

LightGBM is a fast and efficient machine learning algorithm used for classification and regression. It builds decision trees using a gradient boosting approach. It is designed to handle large datasets with high speed and accuracy.



4.10.8 Logistic Regression

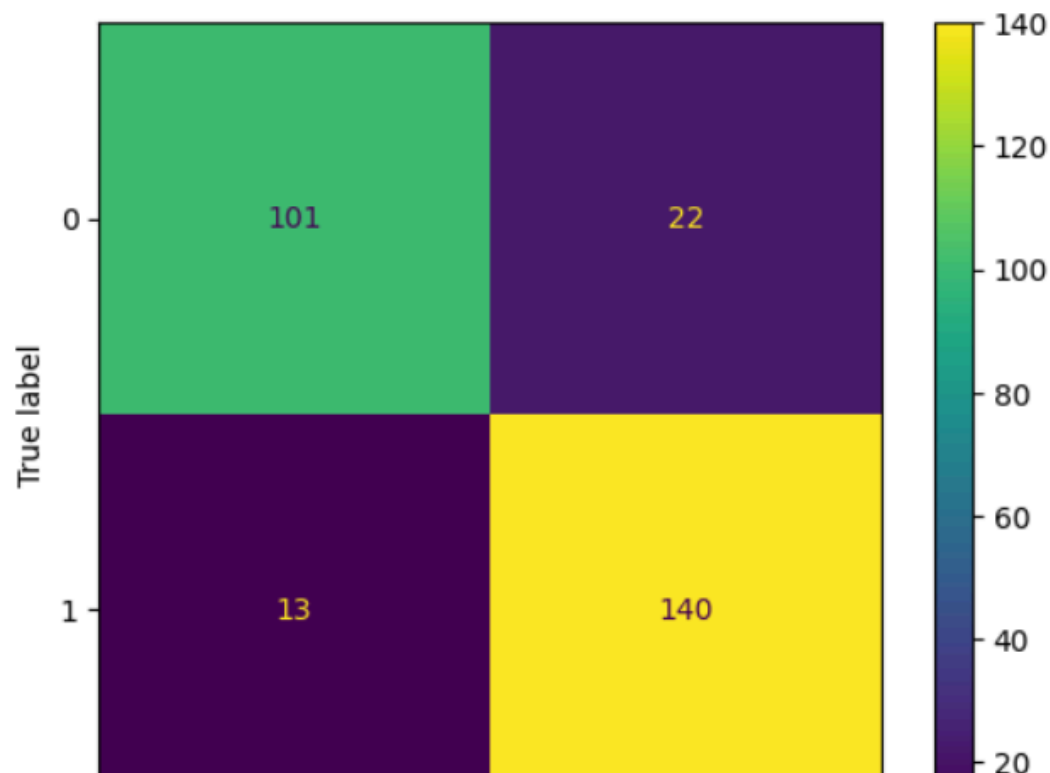
Logistic Regression is a machine learning algorithm used for classification. It predicts the probability of an outcome (usually yes/no). It is simple, fast, and commonly used for binary classification problems.

```
Accuracy: 0.8731884057971014
Precision: 0.8641975308641975
Recall: 0.9150326797385621
F1 Score: 0.8888888888888888
```

Confusion Matrix

Classification Report

	precision	recall	f1-score	support
0	0.89	0.82	0.85	123
1	0.86	0.92	0.89	153
accuracy			0.87	276
macro avg	0.88	0.87	0.87	276
weighted avg	0.87	0.87	0.87	276



4.10.9 Linear Discriminant Analysis

LDA (Linear Discriminant Analysis) is a machine learning algorithm used for classification and dimensionality reduction. It separates data by finding a linear combination of features. It works well when classes are clearly separable.

Accuracy: 0.8876811594202898

Precision: 0.88125

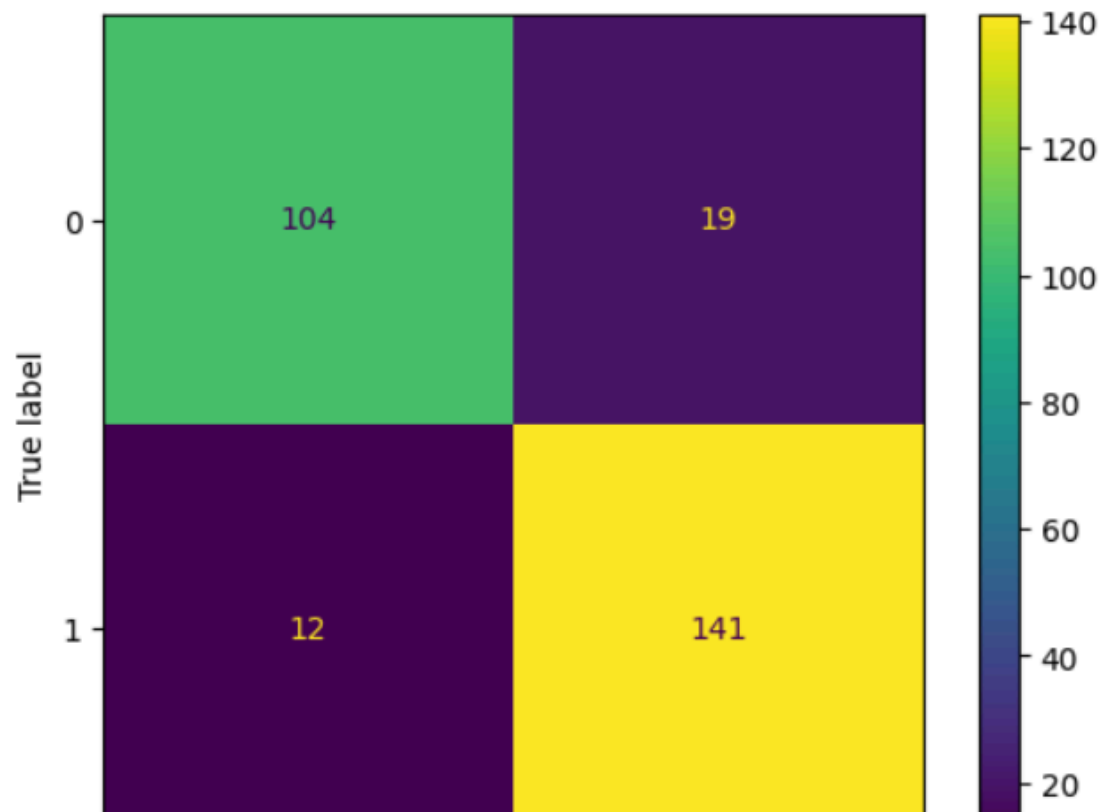
Recall: 0.9215686274509803

F1 Score: 0.9009584664536742

Confusion Matrix

Classification Report

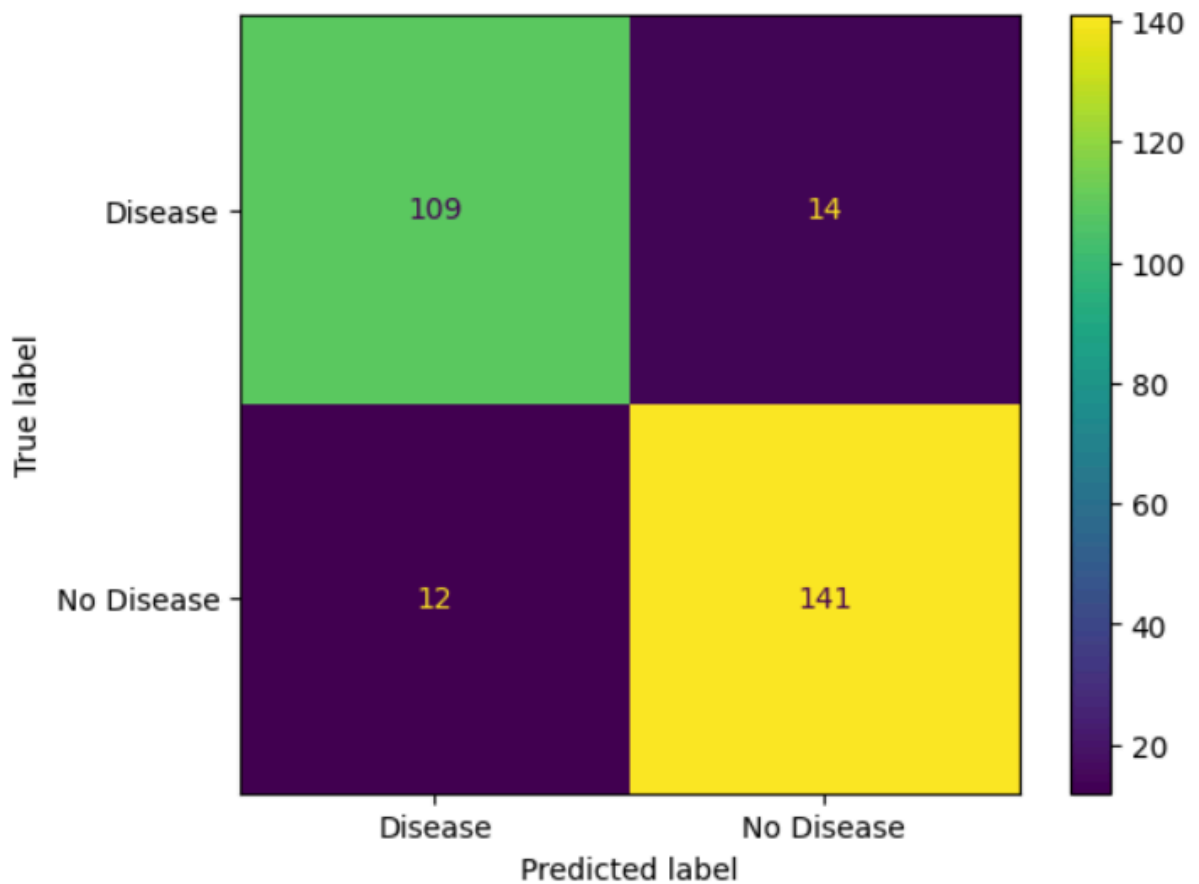
	precision	recall	f1-score	support
0	0.90	0.85	0.87	123
1	0.88	0.92	0.90	153
accuracy			0.89	276
macro avg	0.89	0.88	0.89	276
weighted avg	0.89	0.89	0.89	276



4.10.10 CATBOOST

CatBoost is a machine learning algorithm used for classification and regression. It builds decision trees using a boosting technique. It works well with categorical data and gives high accuracy with less tuning.

```
255.      train: 0.1515022      total: 2.173      remaining: 0.03  
Accuracy: 0.9057971014492754  
Precision: 0.9096774193548387  
Recall: 0.9215686274509803  
F1 Score: 0.9155844155844156  
ROC AUC: 0.9038737446197991  
Confusion_matrix  
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1b1a39f5150>
```



MODEL PERFORMANCE COMPARISON

Models	Base Paper Accuracy	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8841	0.8731	0.8641	0.9150	0.8889	0.8680
SVM	0.8841	0.8841	0.9074	0.8693	0.9108	0.9221
KNN	0.8841	0.8659	0.8717	0.8889	0.8802	0.9324
Random Forest	0.8877	0.8840	0.8757	0.9215	0.8980	0.8794
XgBoost	0.8297	0.9021	0.9090	0.9150	0.9120	0.95245
AdaBoost	0.8659	0.8876	0.9066	0.8888	0.8976	0.8875
Extra Tree	0.8804	0.8442	0.8481	0.8758	0.8617	0.8403
LDA	0.8696	0.8876	0.8812	0.9215	0.9009	0.8680
LightGBM	0.8732	0.8949	0.9133	0.8954	0.9042	0.8948
Gradient Boosting	0.8768	0.9094	0.9155	0.9215	0.9185	0.9079
CatBoost	0.9094	0.9057	0.9096	0.9215	0.9155	0.9038

CHAPTER 5

CONCLUSION AND FUTURE PLANS

5.1 CONCLUSION

Several machine learning models have been implemented to predict heart disease using clinical data. Models such as Logistic Regression, Random Forest, and Support Vector Machine etc were trained and evaluated. After comparing their performance, CatBoost achieved the highest accuracy and gave better prediction results compared to other models. Hence, CatBoost was selected as the final model for heart disease prediction. CatBoost-based machine learning model outperformed several existing state-of-the-art approaches in classifying and predicting cardiovascular disease. The study demonstrated that machine learning can serve as a valuable supplement to clinical practice, enhancing early diagnosis and reducing the need for expensive diagnostic procedures, thereby lowering the financial burden on both patients and the healthcare system.

5.2 FUTURE PLANS

In future work, the performance of the model can be improved by using advanced ensemble techniques and optimizing hyperparameters. The project can be further extended by integrating deep learning models and other architectures to effectively handle complex and large-scale data. Additionally, applying feature engineering and feature selection techniques can help enhance the overall accuracy of the model. Increasing the dataset size and incorporating real-time data will make the system more robust and practical for real-world applications. The model can also be deployed as a web or mobile application to enable easy user interaction. Furthermore, implementing model explainability techniques such as SHAP will help in better understanding the predictions made by both ensemble and deep learning models. Exploring hybrid models that combine ensemble methods with deep learning approaches can further improve performance.

CHAPTER 6

REFERENCES

- [1] .Ensemble learning with explainable AI for improved heart disease prediction based on multiple datasets Shahid Mohammad Ganie,Pijush K anti Dutta Pramanik & Zhongming Zhao(2025) - <https://www.nature.com/articles/s41598-025-97547-6>.
- [2]. Heart Risk Failure Prediction Using a Novel Feature Selection Method for Feature Refinement and Neural Network for Classification Ashir Javeed, Sanam Shahla Rizvi Shijie Zhou, Rabia Riaz, Shafqat Ullah Khan, Se Jin Kwon(2020)
<https://onlinelibrary.wiley.com/doi/10.1155/2020/8843115>
- [3] .A Heart Disease Prediction Model using SVM-Decision Trees-Logistic Regression (SDL) by Mythili T., Dev Mukherji, Nikita Padalia Abhiram Naidu(2020)
<https://www.ijcaonline.org/archives/volume68/number16/11662-7250/>
- [4] Explainable AI assisted heart disease diagnosis through effective feature engineering and stacked ensemble learning panelPartho Ghose d, Khondokar Oliullah a, Md Kawsher Mahbub b, Milon Biswas c, Kazi Nur Uddin b, Hasan M. Jamil (2025)
<https://www.sciencedirect.com/science/article/abs/pii/S0957417424027957>
- [5].An Explainable and Accurate Machine Learning Approach for Early Heart Disease Prediction Using Feature Selection and Ensemble TechniquesAuthorsKhaliq Ahmed(2025) <https://abbdm.com/index.php/Journal/article/view/335>

[6].Medical decision making diagnosis system integrating k-means and Naïve Bayes algorithms Aigerim Altayeva; Suleimenov Zharas; Young Im Cho(2020)
<https://ieeexplore.ieee.org/document/7832446> .